Report for Assignment 7: Picking-Time Neural Network

Julian Soh

Department of Mechanical Engineering, Saint Martin's University
ME/EE 467—Machine Intelligence

April 5, 2026

Abstract. Build and train a feedforward neural network in PyTorch to predict warehouse picking times from operational features. The dataset contains non-linear interactions that a linear model cannot capture. This experiment demonstrates when and why neural networks outperform linear models by examining the Cross-Validation Loss Curves, Predicted-vs.-Actual scatter plots, RMSE comparison, and a depth experiment using PyTorch.

1 Introduction

Linear models cannot effectively learn non-linear data patterns. In contrast, feedforward neural networks can learn non-linear relationships between features and targets. This report demonstrates the superiority of feedforward neural networks over linear models in predicting warehouse picking times from operational features.

A dataset containing non-linear interactions is generated. This dataset contains patterns that a linear model cannot capture, while a feedforward neural network can learn these complex relationships and make accurate predictions.

2 Methods

2.1 Neural Network Training and Cross-Validation

We used a 5-fold cross-validation approach. We held 20% of the data as a validation set and trained the model on the remaining 80%. This process was repeated 5 times, with each fold serving as the validation set once. The training was performed for 200 epochs per fold, and the average validation loss across all folds was computed to evaluate model performance.

After cross-validation, we retrained on the full CV pool (all 5 folds combined) with the same hyperparameters and evaluated the final RMSE.

2.2 Linear Regression

Using Linear Regression, we used the same normalized data as the neural network. This will serve as our comparison of its performance to the neural network.

2.3 Evaluation Metrics

We plotted the Cross-Validation (CV) loss curves for each fold to visualize the learning process. We also created scatter plots of the predicted vs. actual picking times for both the linear model and the neural network to visually assess prediction quality. Finally, we computed and compared the RMSE for both models on the test set. Collectively, we will use these metrics to demonstrate the superiority of the neural network over the linear model in capturing the underlying data patterns and making accurate predictions.

3 Results and Discussion

3.1 Cross-Validation Loss Curves

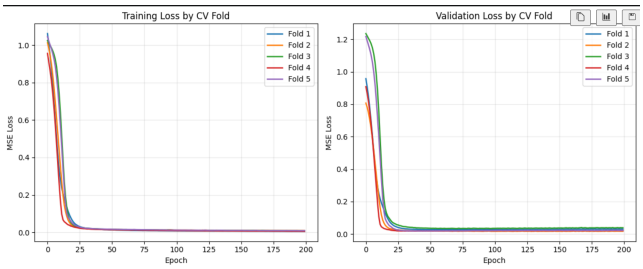

Figure 1: Crosss-Validation Loss Curves for 5 folds.

The two plots shown in Figure 1 illustrate how the neural network learns during **5-fold cross-validation**.

- The **left plot** shows the **training loss** for each fold.
- The **right plot** shows the **validation loss** for each fold.

Loss measures how wrong the model is. A **lower loss** means the predictions are closer to the true picking times. Because this is a regression problem, the loss shown here is the **mean squared error (MSE)**, which penalizes larger mistakes more strongly.

Each colored line corresponds to one fold of cross-validation. In each fold, the model is trained on four subsets of the data and evaluated on the remaining held-out subset. This approach allows us to assess whether performance is consistent across different train/validation splits rather than relying on a single split.

These plots reveal several important observations:

- **The loss drops very quickly in the first few epochs.** This indicates that the network learns the main structure of the problem early in training.

- **The training loss becomes very small for all folds.** This shows that the model is capable of fitting the training data well.
- **The validation loss also drops quickly and remains low.** This is important because it suggests the model is not merely memorizing the training data, but is also performing well on unseen validation examples.
- **The fold-to-fold curves are similar, but not identical.** This variation is expected, since each fold uses a slightly different subset of the data. The similar overall shapes of the curves suggest consistent model behavior across splits.
- **There is no strong evidence of overfitting.** If overfitting were severe, the training loss would continue to decrease while the validation loss would increase or remain substantially higher. Instead, the validation loss stabilizes at a low level after the initial decrease.

Overall, these results indicate that the neural network trains stably, converges quickly, and generalizes well across the different cross-validation folds. This interpretation is consistent with the reported cross-validation RMSE values, which are relatively stable from fold to fold.

3.2 Predicted versus Actual

The two plots in Figure 2 illustrate the same concept for two different models. For each test example, the horizontal axis represents the **actual picking time**, while the vertical axis represents the model's **predicted picking time**.

The dashed diagonal line corresponds to **perfect predictions**. A point lying exactly on this line indicates that the model predicted the picking time correctly. The farther a point deviates from the diagonal, the larger the prediction error.

The plot for the **linear model** shows a clear overall trend, indicating that the model has learned some useful structure in the data. As the actual picking time increases, the predicted picking time also tends to increase. However, many points lie farther from the diagonal, particularly at the lower and higher ends of the range. This indicates that while the linear model captures the general direction of the relationship, it fails to model the full shape of the underlying function accurately.

A particularly important feature of the linear model plot is the presence of **systematic errors** rather than purely random noise. For larger actual picking times, many points fall **below** the diagonal, indicating that the model is consistently **under-predicting** slow or difficult cases. In other words, as task complexity increases, the linear model tends to produce predictions that are too low. This behavior is consistent with the fact that the true data-generating process includes nonlinear effects that a simple linear model cannot represent.

The plot for the **neural network** is noticeably tighter around the diagonal. This indicates that its predictions are generally closer to the true values. Although the points do not lie perfectly on the diagonal—due to noise in the data and the inherent limitations of any model—the overall scatter is reduced compared to the linear model. As a result,

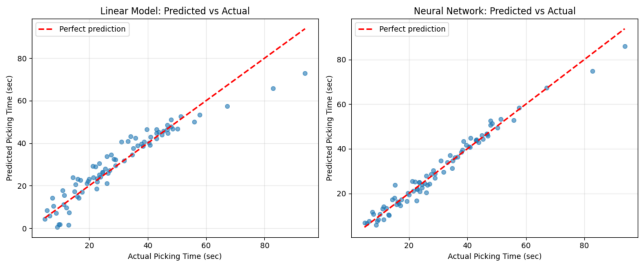

Figure 2: Predicted vs Actual picking times for the final model.

Model	Test RMSE (sec)	Relative	Improvement (%)
Linear Regression	5.45	1.000	0.00
Neural Network	2.72	0.499	50.12

Table 1: Test RMSE comparison between linear regression and neural network models.

the neural network produces smaller prediction errors on average.

The improvement offered by the neural network is most apparent in regions where the linear model struggles:

- **At higher picking times**, the neural network remains closer to the diagonal, indicating better handling of more difficult cases.
- **At the extremes of the range**, the neural network exhibits less systematic bias than the linear model.
- **Across the middle range**, both models perform reasonably well, but the neural network remains more tightly clustered around the ideal prediction line.

In plain terms, this comparison shows that while the linear model captures the broad trend in the data, it is too limited to represent interaction effects, threshold behaviors, and curvature. The neural network, by contrast, is better able to model these nonlinear patterns, resulting in predictions that align more closely with the actual outcomes.

This visual assessment is consistent with the quantitative results: the neural network achieves a lower RMSE than the linear model. Thus, the improved scatter plot is not merely cosmetic, but reflects genuinely superior predictive accuracy.

The **linear-vs.-neural network RMSE comparison** confirms the visual results (Figure 2) quantitatively, the neural network achieves a lower test RMSE than ordinary least squares.

This outcome is expected because the linear baseline can only learn an *affine* relationship in the original input space. In contrast, a multilayer neural network can represent nonlinear functions through compositions of linear transformations and nonlinear activation functions.

3.3 Depth Experiment Curves

The **depth experiment curves** shown in Figure 3 add a more detailed picture. The mean validation-loss curves for 1, 2, and 3 hidden layers show how model depth affects optimization and validation behavior over training epochs.

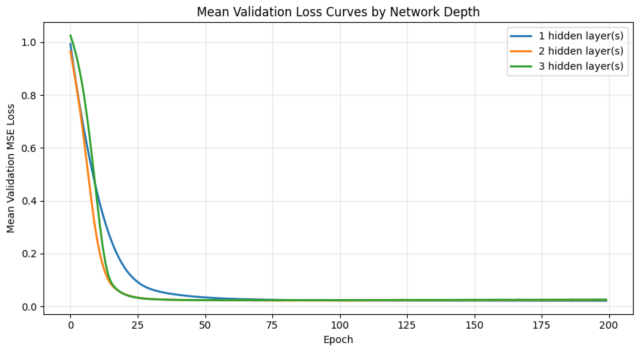

Figure 3: Test RMSE for different model depths.

In the accompanying Jupyter notebook with the experiment, we see that deeper models do not automatically guarantee better performance; instead, the experiment shows that architecture choice should be validated empirically. This is consistent with the discussion that additional depth can improve expressive power, but its practical benefit depends on the task and data.

3.4 Neural Network improvement over Linear

The neural network improves most over the linear model in the parts of the input space where the target function is explicitly non-linear:

- **High distance with high congestion**: the target includes an interaction term ('congestion * distance'), which a plain linear model cannot represent without manually engineered interaction features.
- **Low battery values**: the target includes a threshold-style penalty when 'battery < 0.2', which is not naturally captured by a linear model.
- **Larger distances**: the quadratic distance term introduces curvature, again favoring a nonlinear model.

4 Conclusion

Taken together, the CV loss curves, the predicted-vs.-actual scatter plots, the RMSE comparison, and the depth experiment all support the same conclusion: the neural network is better suited than linear regression for this warehouse picking-time problem because the underlying data is not purely linear.

References

Rico A.R. Picone. *Engineering Artificial Intelligence*. Self-published, 2026.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach, fourth edition*. Pearson, 2021. ISBN 1292401133.